

XML

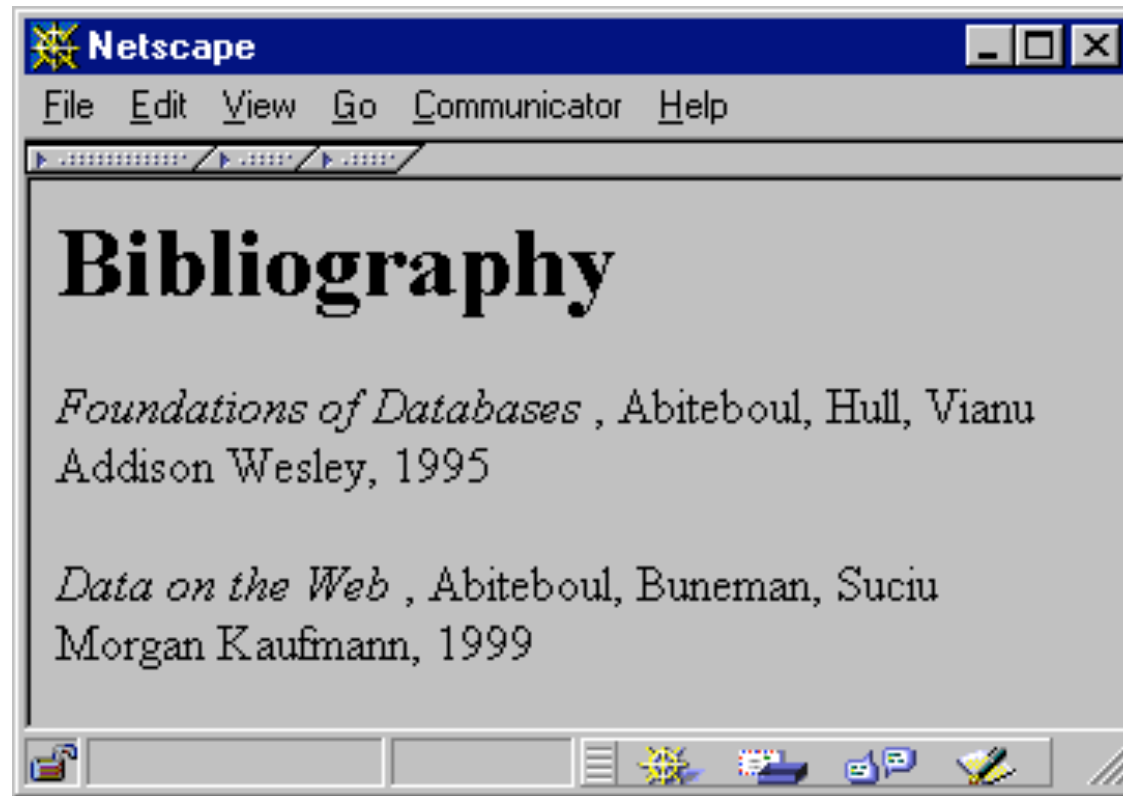
- A flexible syntax for data
- Used in:
 - Configuration files, e.g. Web.Config
 - Replacement for binary formats (MS Word)
 - Document markup: e.g. XHTML
 - Data: data exchange, semistructured data
- Roots: SGML - a very nasty language

We will study only XML as data

XML as Semistructured Data

- Relational databases have rigid schema
 - Schema evolution is costly
- XML is flexible: semistructured data
 - Store data in XML
- Warning: not normal form ! Not even 1NF

From HTML to XML



HTML describes the presentation

HTML

`<h1> Bibliography </h1>`

`<p> <i> Foundations of Databases </i>`

Abiteboul, Hull, Vianu

`<br>` Addison Wesley, 1995

`<p> <i> Data on the Web </i>`

Abiteoul, Buneman, Suciu

`<br>` Morgan Kaufmann, 1999

XML Syntax

```
<bibliography>  
  <book>    <title> Foundations... </title>  
             <author> Abiteboul </author>  
             <author> Hull </author>  
             <author> Vianu </author>  
             <publisher> Addison Wesley </publisher>  
             <year> 1995 </year>  
  </book>  
  ...  
</bibliography>
```

XML describes the content

XML Terminology

- tags: `book`, `title`, `author`, ...
- start tag: `<book>`, end tag: `</book>`
- elements: `<book>...</book>`, `<author>...</author>`
- elements are nested
- empty element: `<red></red>` abbrev. `<red/>`
- an XML document: single *root element*

well formed XML document: if it has matching tags

More XML: Attributes

```
<book price = "55" currency = "USD">  
  <title> Foundations of Databases </title>  
  <author> Abiteboul </author>  
  ...  
  <year> 1995 </year>  
</book>
```

Attributes v.s. Elements

```
<book price = "55" currency = "USD">  
  <title> Foundations of DBs </title>  
  <author> Abiteboul </author>  
  ...  
  <year> 1995 </year>  
</book>
```

```
<book>  
  <title> Foundations of DBs </title>  
  <author> Abiteboul </author>  
  ...  
  <year> 1995 </year>  
  <price> 55 </price>  
  <currency> USD </currency>  
</book>
```

attributes are alternative ways to represent data

Comparison

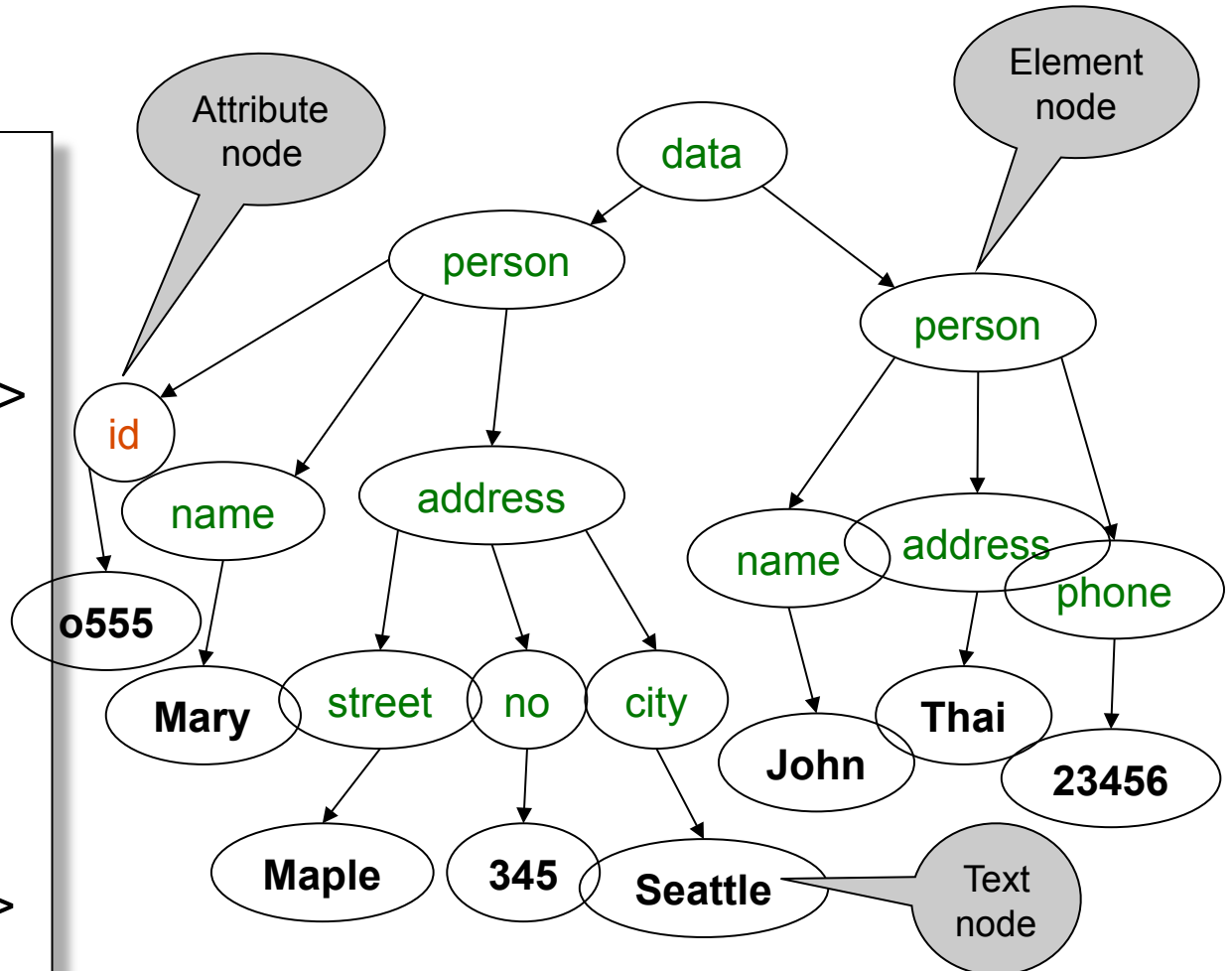
Elements	Attributes
Ordered	Unordered
May be repeated	Must be unique
May be nested	Must be atomic

That's All !

- That's all you ever need to know about XML syntax
 - Optional type information can be given in the DTD or XSchema (later)
 - We'll discuss some additional syntax in the next few slides, but that's not essential
- What is important for you to know: XML's semantics

XML Semantics: a Tree !

```
<data>
  <person id="o555" >
    <name> Mary </name>
    <address>
      <street>Maple</street>
      <no> 345 </no>
      <city> Seattle </city>
    </address>
  </person>
  <person>
    <name> John </name>
    <address>Thailand
    </address>
    <phone>23456</phone>
  </person>
</data>
```



Order matters !!!

XML as Data

- XML is self-describing
- Schema elements become part of the data
 - Relational schema: `persons(name,phone)`
 - In XML `<persons>`, `<name>`, `<phone>` are part of the data, and are repeated many times
- Consequence: XML is much more flexible
- XML = semistructured data

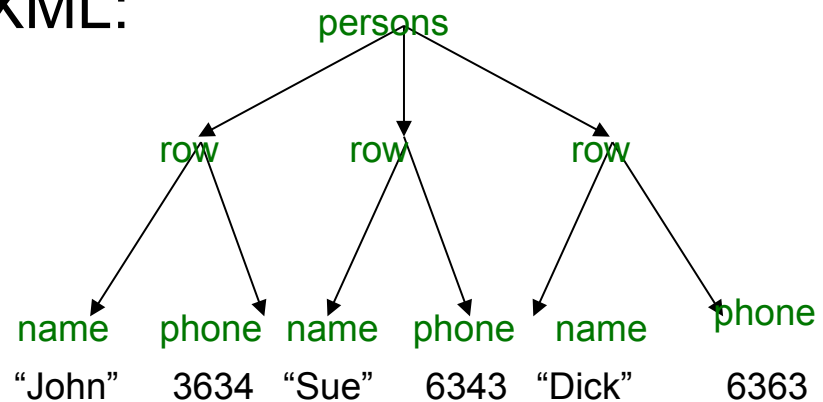
Mapping Relational Data to XML

The canonical mapping:

Persons

Name	Phone
John	3634
Sue	6343
Dick	6363

XML:



```
<persons>
  <row> <name>John</name>
    <phone> 3634</phone></row>
  <row> <name>Sue</name>
    <phone> 6343</phone>
  <row> <name>Dick</name>
    <phone> 6363</phone></row>
</persons>
```

Mapping Relational Data to XML

Natural mapping

Persons

Name	Phone
John	3634
Sue	6343

Orders

PersonName	Date	Product
John	2002	Gizmo
John	2004	Gadget
Sue	2002	Gadget

XML

```
<persons>
  <person>
    <name> John </name>
    <phone> 3634 </phone>
    <order> <date> 2002 </date>
              <product> Gizmo </product>
    </order>
    <order> <date> 2004 </date>
              <product> Gadget </product>
    </order>
  </person>
  <person>
    <name> Sue </name>
    <phone> 6343 </phone>
    <order> <date> 2004 </date>
              <product> Gadget </product>
    </order>
  </person>
</persons>
```

XML is Semi-structured Data

- Missing attributes:

```
<person>  <name> John</name>  
           <phone>1234</phone>  
</person>  
  
<person>  <name>Joe</name>  
</person>
```

no phone !

- Could represent in
a table with nulls

name	phone
John	1234
Joe	-

XML is Semi-structured Data

- Repeated attributes

```
<person> <name> Mary</name>  
          <phone>2345</phone>  
          <phone>3456</phone>  
</person>
```

Two phones !

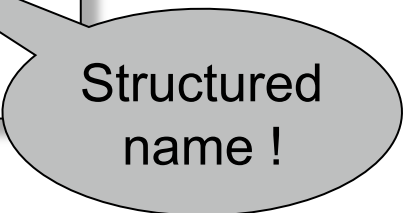
- Impossible in tables:

name	phone		
Mary	2345	3456	???

XML is Semi-structured Data

- Attributes with different types in different objects

```
<person> <name> <first> John </first>  
                <last> Smith </last>  
            </name>  
            <phone>1234</phone>  
</person>
```



Structured
name !

- Nested collections (no 1NF)
- Heterogeneous collections:
 - <db> contains both <book>s and <publisher>s

Querying XML Data

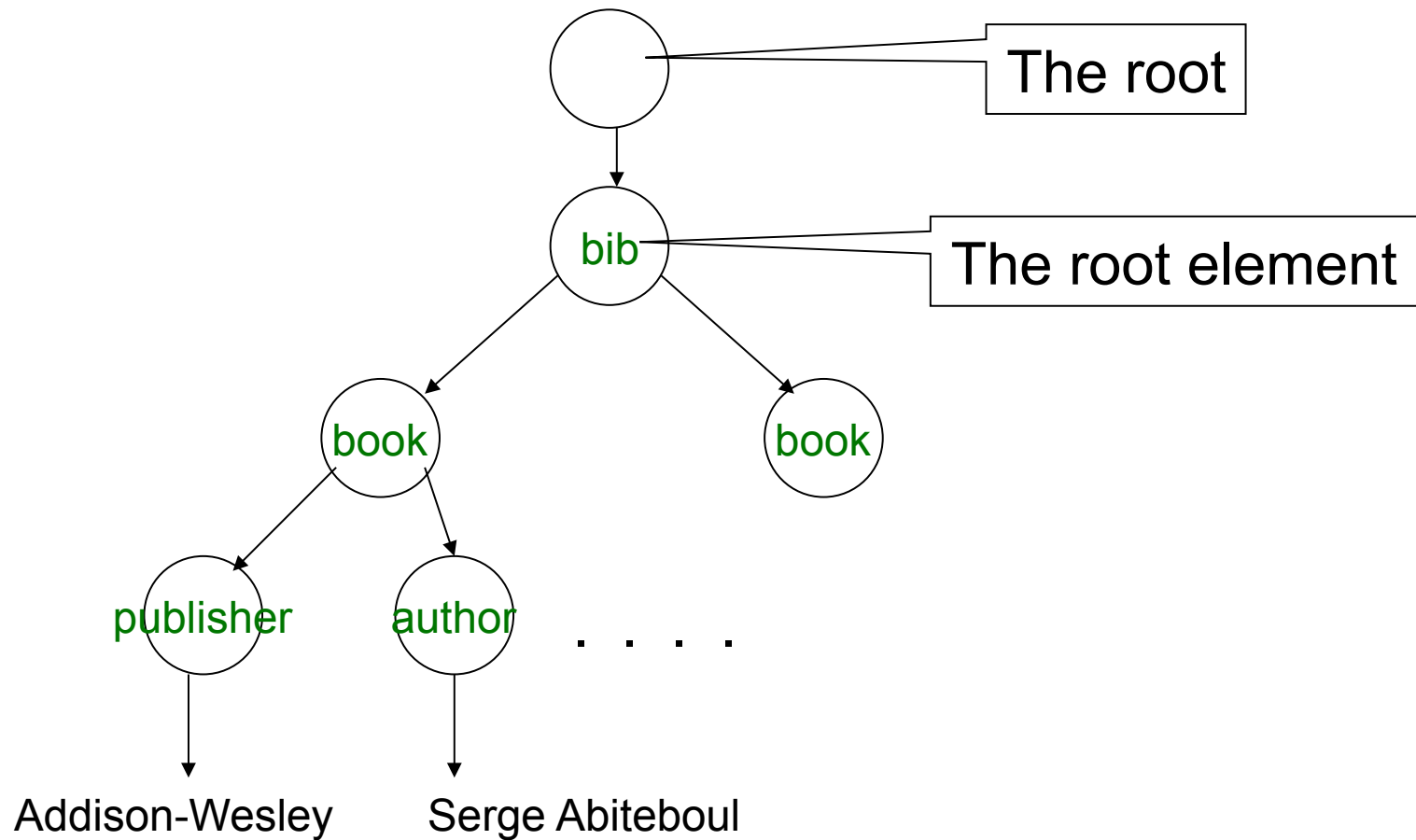
Will discuss next:

- XPath = simple navigation on the tree
- XQuery = “the SQL of XML”

Sample Data for Queries

```
<bib>
  <book> <publisher> Addison-Wesley </publisher>
    <author> Serge Abiteboul </author>
    <author> <first-name> Rick </first-name>
      <last-name> Hull </last-name>
    </author>
    <author> Victor Vianu </author>
    <title> Foundations of Databases </title>
    <year> 1995 </year>
  </book>
  <book price="55">
    <publisher> Freeman </publisher>
    <author> Jeffrey D. Ullman </author>
    <title> Principles of Database and Knowledge Base Systems </title>
    <year> 1998 </year>
  </book>
</bib>
```

Data Model for XPath



XPath: Simple Expressions

`/bib/book/year`

Result: `<year> 1995 </year>`
`<year> 1998 </year>`

`/bib/paper/year`

Result: empty (there were no papers)

`/bib`

What's the difference ?

`/`

XPath: Restricted Kleene Closure

`//author`

Result: `<author>` Serge Abiteboul `</author>`
 `<author>` `<first-name>` Rick `</first-name>`
 `<last-name>` Hull `</last-name>`
 `</author>`
 `<author>` Victor Vianu `</author>`
 `<author>` Jeffrey D. Ullman `</author>`

Result: `<first-name>` Rick `</first-name>`

`/bib//first-name`

Xpath: Attribute Nodes

`/bib/book/@price`

Result: “55”

`@price` means that price is has to be an attribute

Xpath: Wildcard

`//author/*`

Result: `<first-name>` Rick `</first-name>`
`<last-name>` Hull `</last-name>`

* Matches any element

@* Matches any attribute

Xpath: Text Nodes

```
/bib/book/author/text()
```

Result: Serge Abiteboul
Victor Vianu
Jeffrey D. Ullman

Rick Hull doesn't appear because he has `firstname`, `lastname`

Functions in XPath:

- `text()` = matches the text value
- `node()` = matches any node (= * or @* or `text()`)
- `name()` = returns the name of the current tag

Xpath: Predicates

```
/bib/book/author[first-name]
```

Result: <author> <first-name> Rick </first-name>
 <last-name> Hull </last-name>
 </author>

Xpath: More Predicates

```
/bib/book/author[first-name][address[.//zip][city]]/last-name
```

Explain how this is evaluated !

Xpath: More Predicates

```
/bib/book/author[first-name][address[./zip][city]]/last-name
```

Result: <lastname> ... </lastname>
<lastname> ... </lastname>

How do we read this ?

First remove all qualifiers (predicates):

```
/bib/book/author/last-name
```

Then add them one by one:

```
/bib/book/author[first-name][address]/last-name
```

Xpath: More Predicates

```
/bib/book[@price < 60]
```

```
/bib/book[author/@age < 25]
```

```
/bib/book[author/text()]
```

Xpath: Brief Summary

<code>bib</code>	matches a <code>bib</code> element
<code>*</code>	matches any element
<code>/</code>	matches the <code>root</code> element
<code>/bib</code>	matches a <code>bib</code> element under <code>root</code>
<code>bib/paper</code>	matches a <code>paper</code> in <code>bib</code>
<code>bib//paper</code>	matches a <code>paper</code> in <code>bib</code> , at any depth
<code>//paper</code>	matches a <code>paper</code> at any depth
<code>paper book</code>	matches a <code>paper</code> or a <code>book</code>
<code>@price</code>	matches a <code>price</code> attribute
<code>bib/book/@price</code>	matches <code>price</code> attribute in <code>book</code> , in <code>bib</code>
<code>bib/book[@price<"55"]/author/lastname</code>	matches...